Process Management

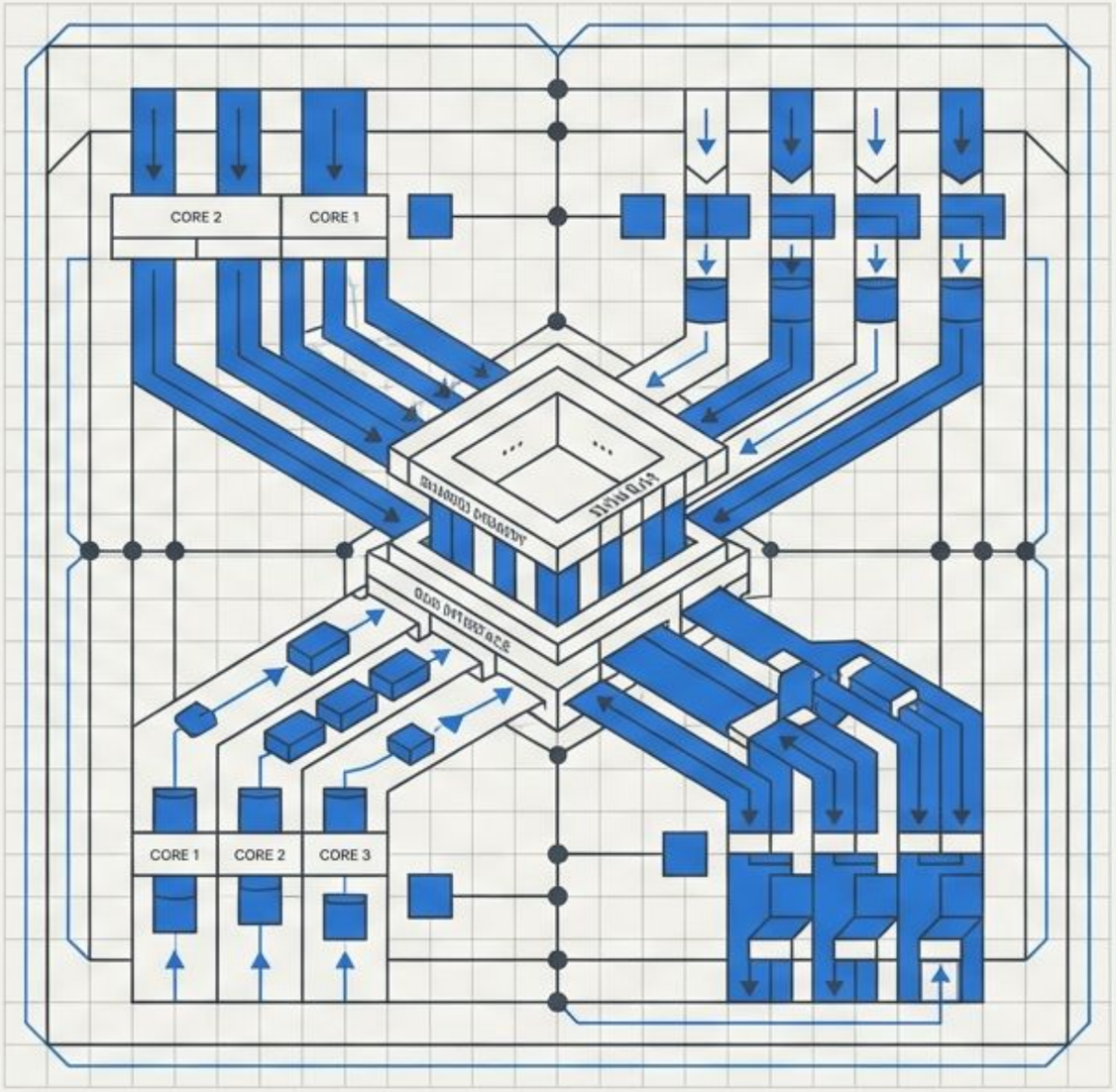
Unit 2: Core Operating System Operations

I. Process Fundamentals & Threads

II. Inter-Process Communication (IPC)

III. Synchronization & Classical Problems

IV. Process Scheduling Strategies



Prepared By : Er. Sharat Maharjan

From Passive to Active: Program vs. Process

The Trigger:
Loaded into Memory

The Program

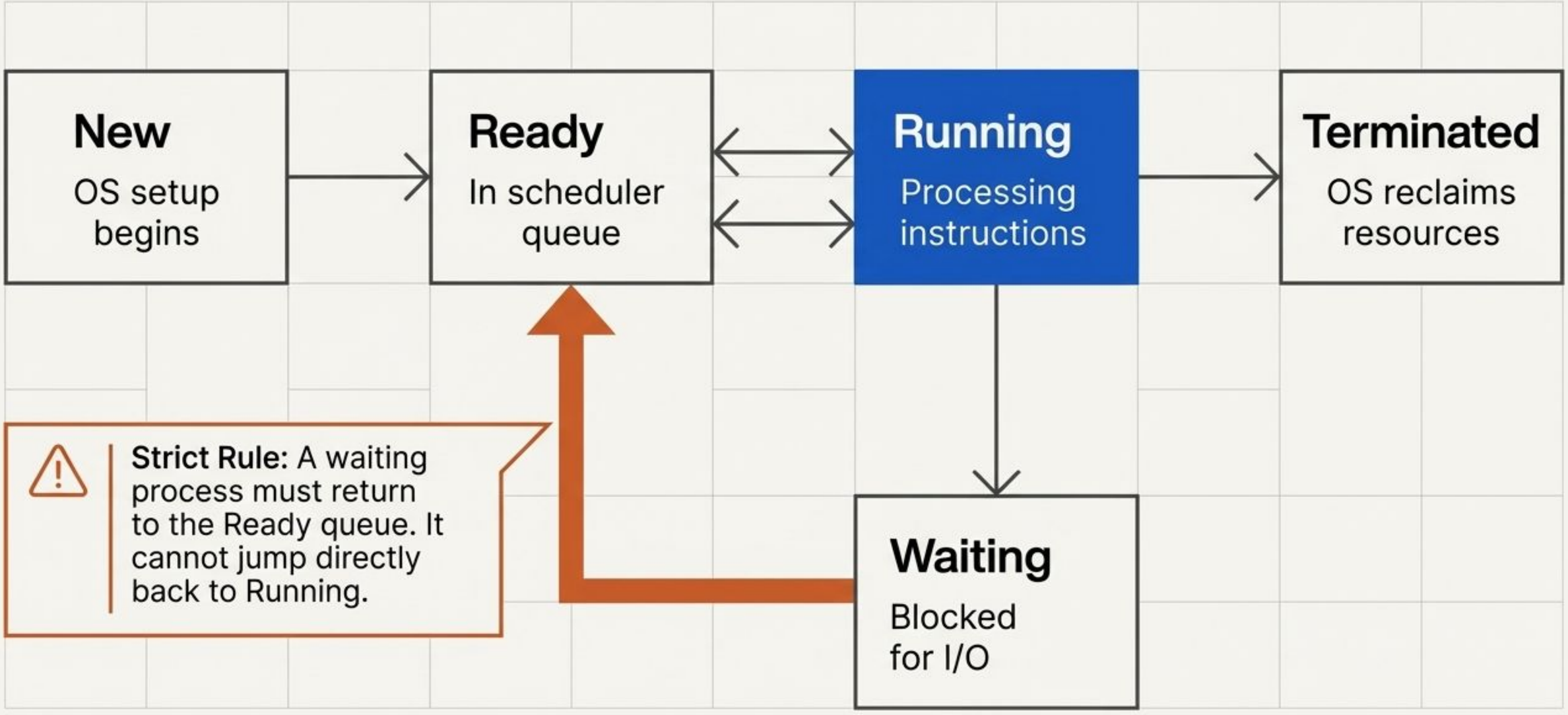
- **State:** Passive Entity
- **Location:** Stored on Disk
- **Nature:** Static executable file
- **Example:** hello.py script

The Process

- **State:** **Active** Entity
- **Location:** Resides in RAM
- **Nature:** **Dynamic** instance with state
- **Example:** python hello.py (Assigned a PID)

Multiprogramming Goal: Run multiple processes concurrently by switching the CPU between them, maximizing utilization during I/O wait times.

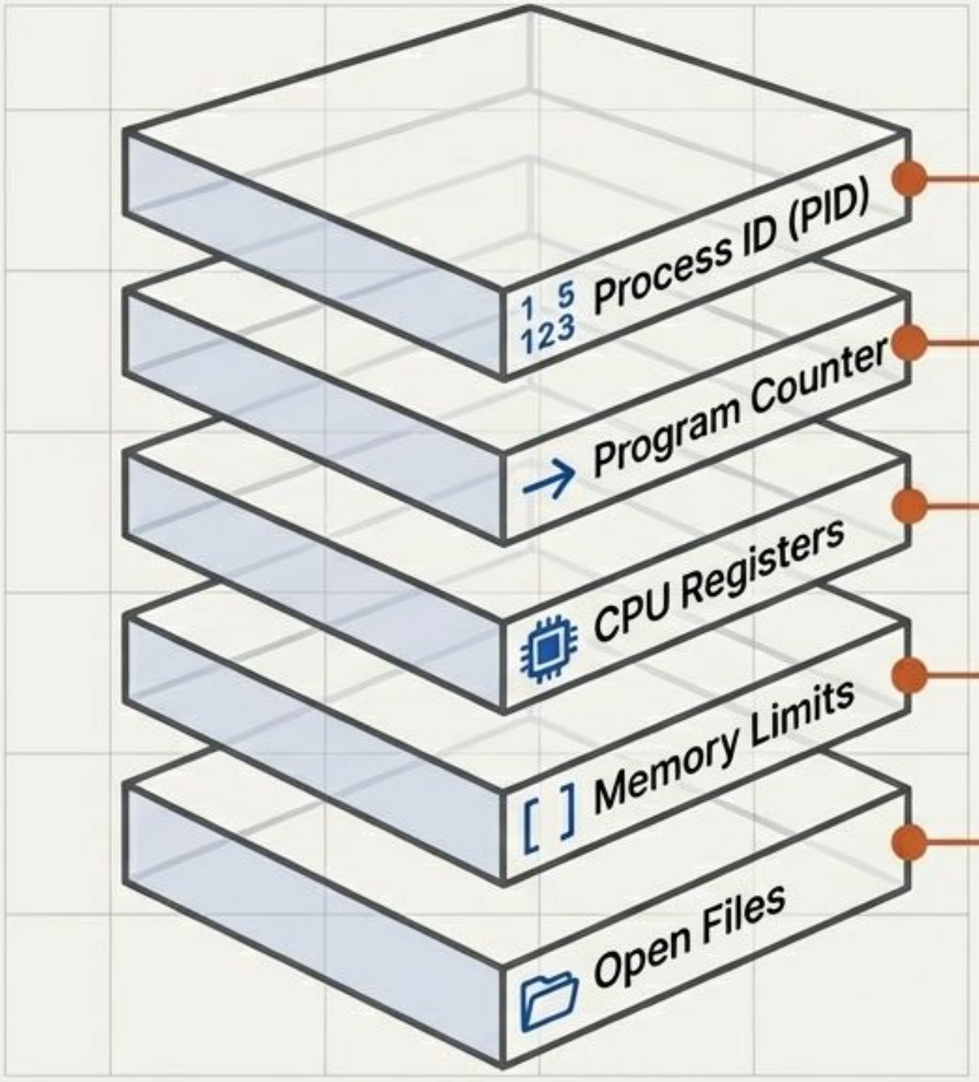
The 5-State Process Lifecycle



Process Control Block (PCB): The Metadata

Definition:
A specialized kernel data structure storing essential metadata.

The OS uses the PCB to pause, save, and perfectly resume an active process.



Unique numerical identifier for OS system tracking.

The exact memory address of the next instruction to execute.

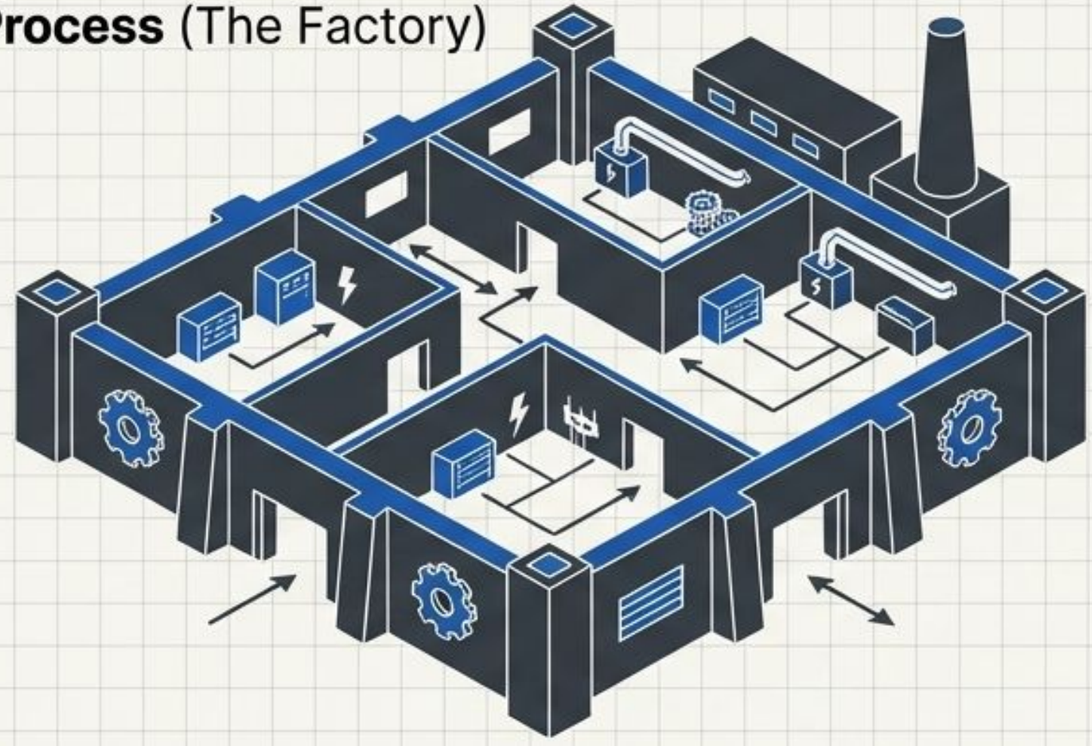
Saved processor state variables required for seamless context switching.

Strict boundaries defining the process's isolated address space.

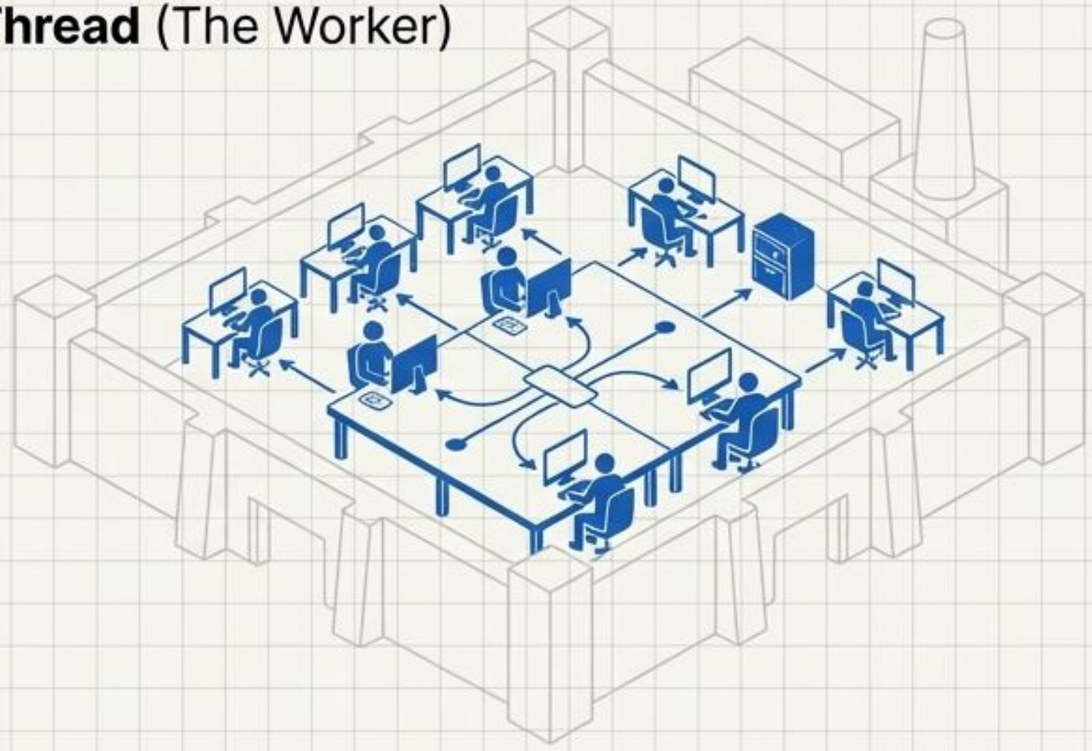
Tracked list of currently accessed I/O resources and file descriptors.

System Architecture: Processes vs. Threads

Process (The Factory)



Thread (The Worker)



- **Memory:** Completely isolated space
- **Memory:** Compietely isolated space
- **Weight:** Heavyweight entity
- **Switching:** Slower context switch
- **Isolation:** Crash is safely contained
- **Comms:** Requires IPC system calls

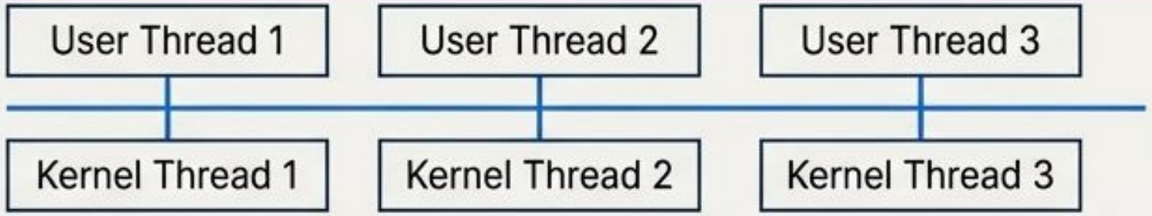
Architectural Advantage: Threads allow simultaneous execution within the same application environment without the heavy overhead of creating entirely new isolated processes.

- **Memory:** Shares memory with parent
- **Weight:** Lightweight entity
- **Switching:** Significantly faster
- **Isolation:** Crash kills entire process
- **Comms:** Direct via shared variables
- **Comms:** Direct via shared variables

Thread Management Models

User-Level Threads (ULT)	Kernel-Level Threads (KLT)
Managed by user-space libraries (e.g., Python threading).	Managed directly by the OS kernel (e.g., Linux pthreads).
PRO: Extremely fast; requires no kernel mode switching.	**PRO:** Enables true multicore parallelism; kernel handles blocking.
CON: Blocking one thread blocks them all. No true parallelism.	**CON:** Slower creation and context switching overhead.

The Hybrid Solution (Linux 1:1 Model)

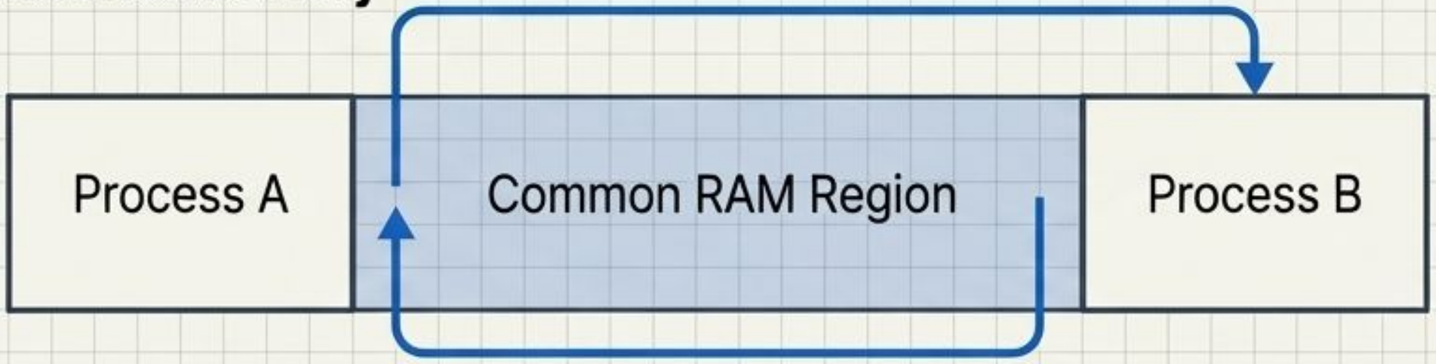


Combines advantages by mapping each user thread directly to a dedicated kernel thread, allowing efficient multicore execution.

Inter-Process Communication (IPC)

Bridging architectural isolation so processes can synchronize and share data.

Shared Memory

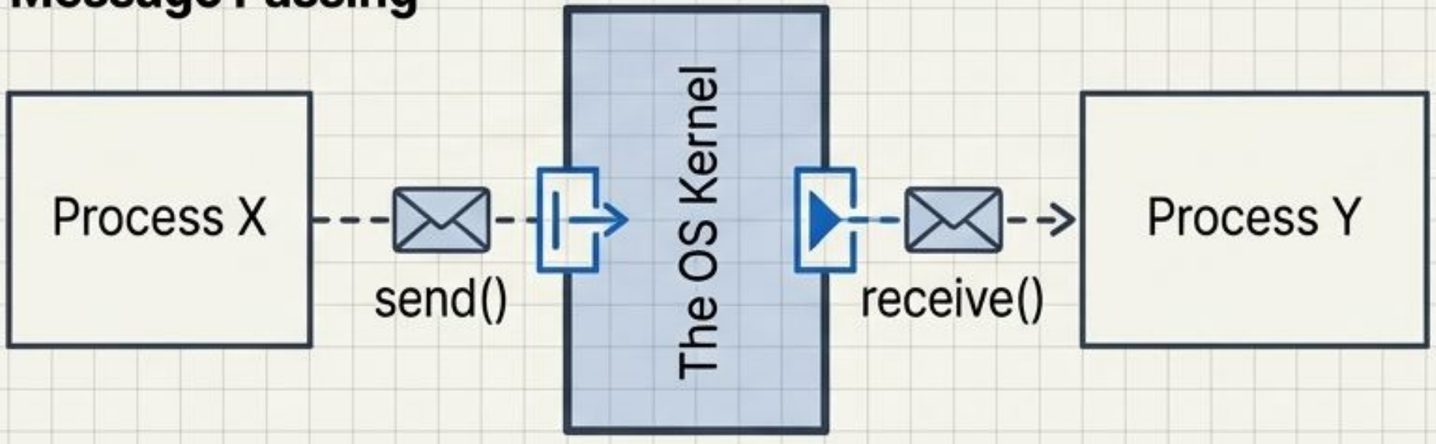


Mechanism: Direct access to shared memory space.

Advantage: The absolute fastest IPC method.

Note: Bypasses OS kernel entirely.

Message Passing



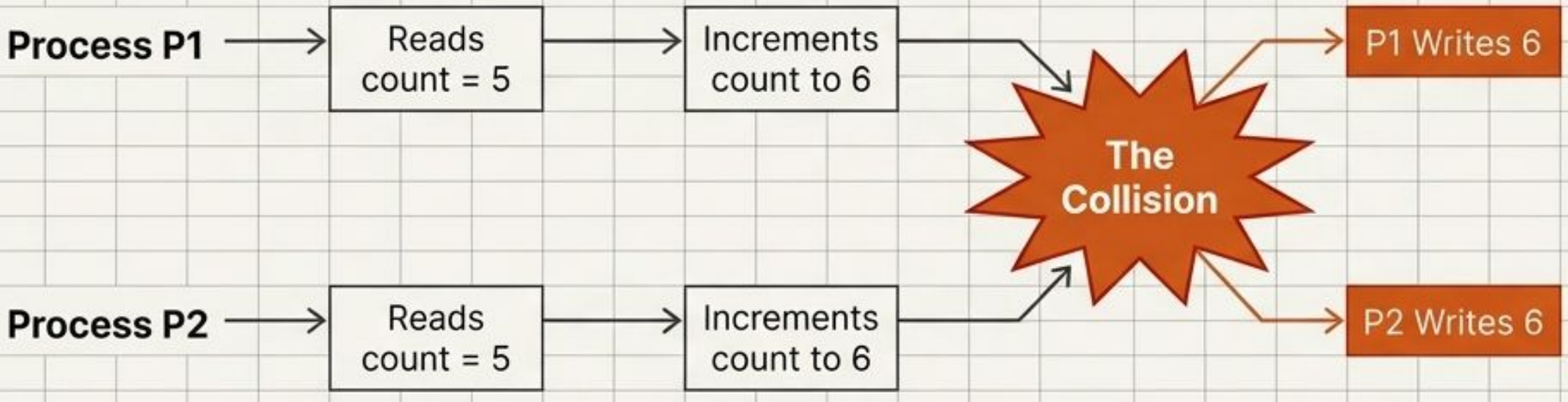
Mechanism: Communication via OS system calls.

Advantage: Ideal for distributed systems and networks.

Note: Slower due to mandatory kernel mediation.

The Concurrency Crisis: Race Conditions

Definition: Concurrent access to shared data where the final outcome depends unpredictably on the exact order order of execution, leading to inconsistent data.



Results

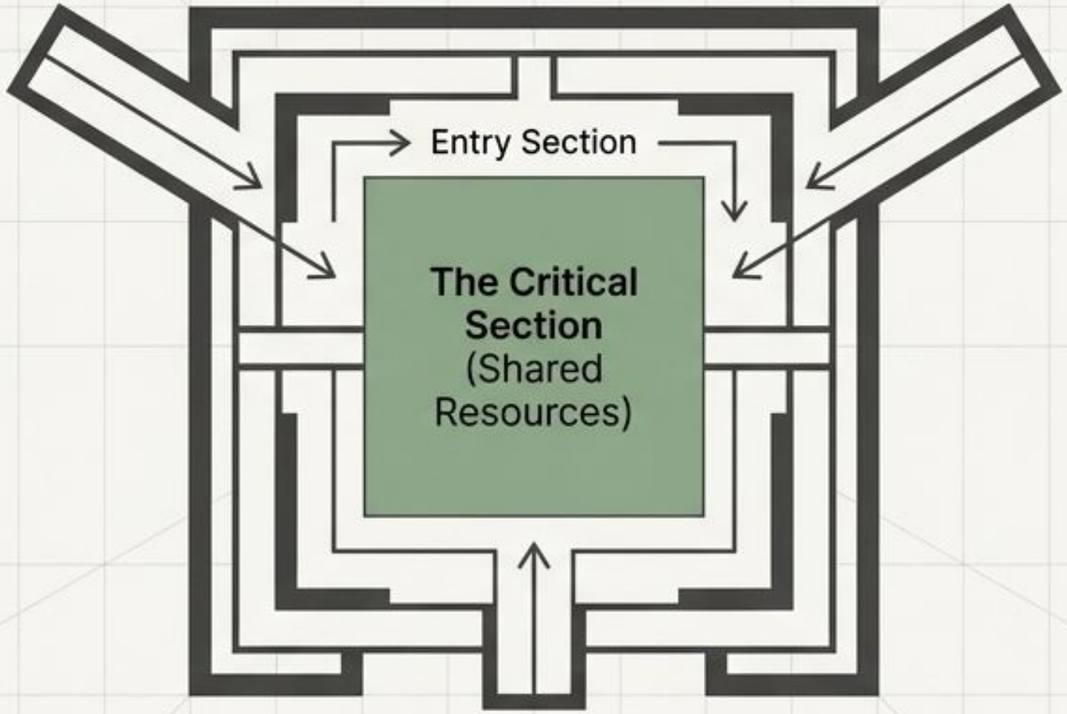
Expected Result:
count = 7
(If sequential)

Actual Result:
count = 6
(The Lost Update)

Defining the Safe Zone: Critical Section

1. Mutual Exclusion

Absolutely only one process may execute inside the critical section at any given time.



2. Progress

If the section is empty, a waiting process must be allowed to enter without delay.

3. Bounded Waiting

No process can be forced to wait indefinitely.
A strict limit guarantees no starvation.

The Target: A valid synchronization system MUST satisfy all three rules simultaneously to definitively eliminate race conditions.

Enforcing Mutual Exclusion: Solution Tactics

Busy Waiting (Spinlocks)

```
while (lock == 1);
```

- Continuous loop checking lock status.
- Severe waste of CPU cycles.
- Hardware solutions: Disabling Interrupts (dangerous) or TSL instructions.
- Software solutions: Peterson's Algorithm (perfect for 2 processes).

Sleep & Wakeup

```
sleep();
wakeup();
```

- OS aggressively suspends waiting processes.
- Conserves CPU power for other tasks.
- Vulnerability: Susceptible to catastrophic "Lost Wakeup" signals if triggered out of order.

Advanced Synchronization: Semaphores

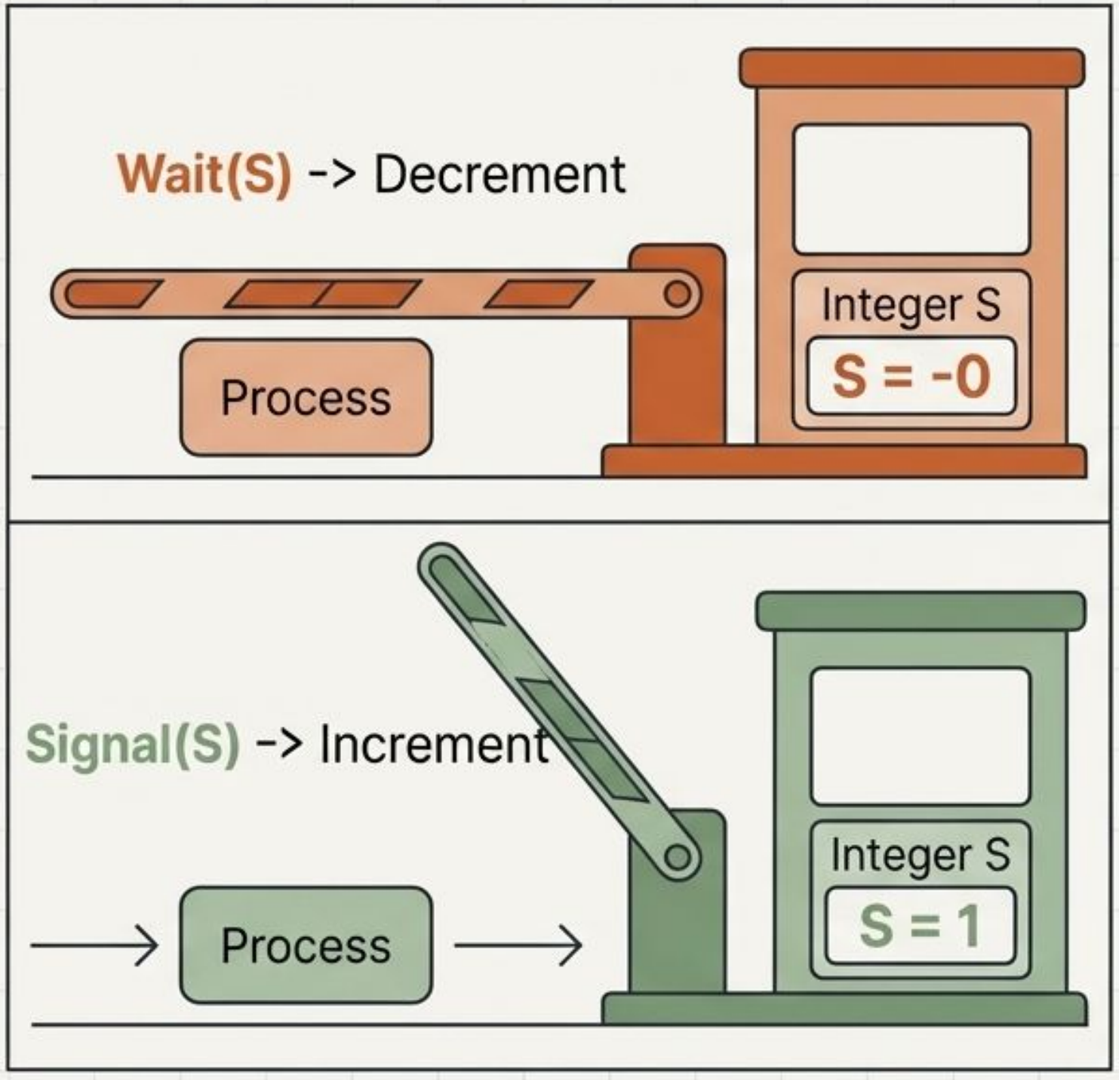
Definition: An atomic integer variable (S) manipulated exclusively via two operations to control structural access.

Wait(S): Decrements S. If S < 0, the process goes to sleep.

Signal(S): Increments S. Wakes up a sleeping process in the queue.

Binary Semaphore: Functions exactly as a Mutex lock (S is strictly 0 or 1).

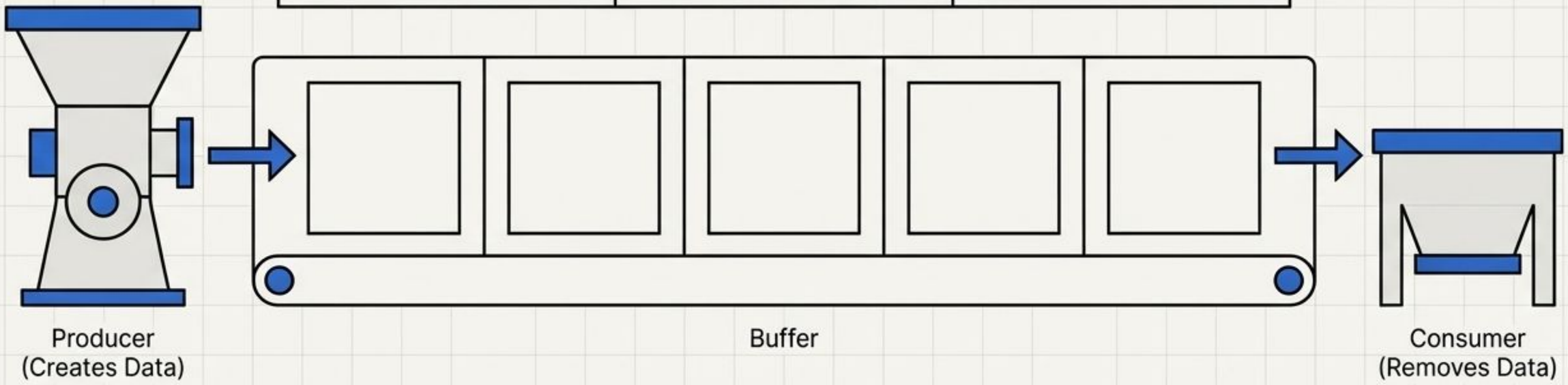
Counting Semaphore: Allows exactly N processes into the section simultaneously.



Classical Problem: Bounded Buffer Protocol

The Producer-Consumer Relationship

mutex = 1	empty = N	full = 0
Protects mutual exclusion	Counts available spaces	Counts items to consume



The Constraints

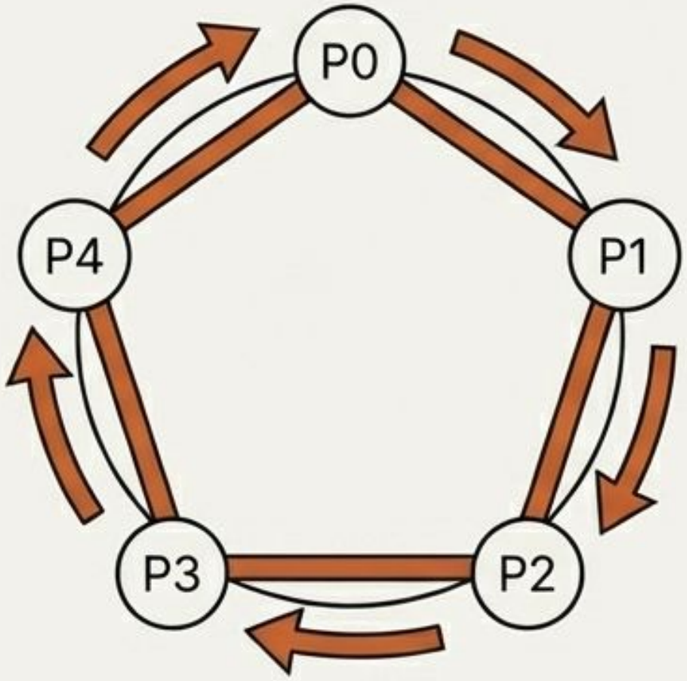
- Producer must wait if the buffer is full.
- Consumer must wait if the buffer is empty.

Execution Flow

- Producer: wait(empty) -> wait(mutex) -> Add -> signal(mutex) -> signal(full)
- Consumer: wait(full) -> wait(mutex) -> Remove -> signal(mutex) -> signal(empty)

Classical Problem: Dining Philosophers & Deadlock

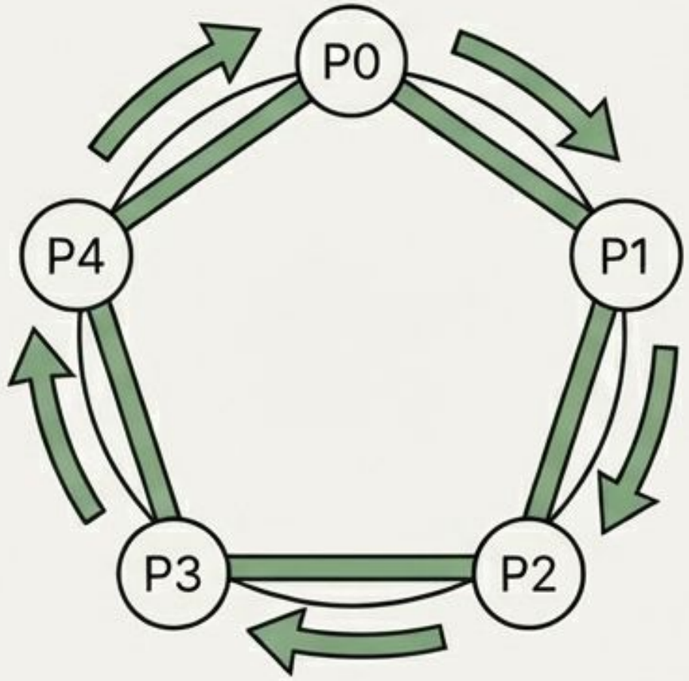
The Threat: Deadlock



Deadlock: Circular wait occurs. If all 5 grab their left fork simultaneously, the system freezes permanently.

Starvation: A philosopher is perpetually bypassed and never gets resources.

The Solution: Asymmetric Strategy



Asymmetric Solution: Breaking the circular wait.
Even philosophers grab left fork first.
Odd philosophers grab right fork first.

Result: Guarantees progress and prevents gridlock.

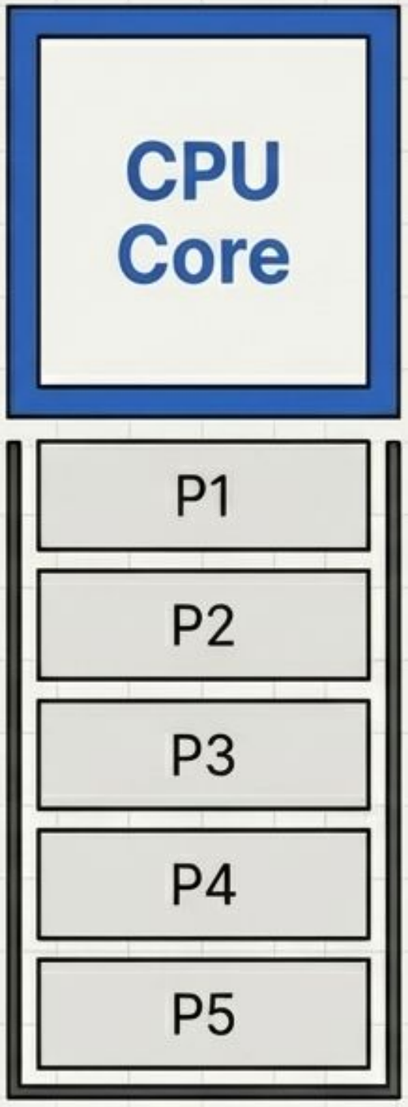
Process Scheduling Strategies

Preemptive

OS can forcefully interrupt and pause a running process.

Non-Preemptive

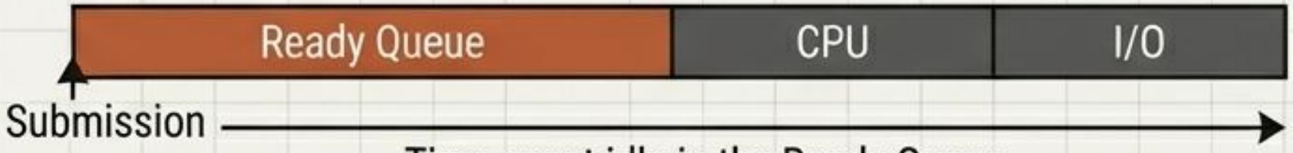
Process retains control until voluntary surrender.



Turnaround Time

Total lifespan from Submission to Completion.

Waiting Time



Time spent idle in the Ready Queue.

Response Time

Time from Submission until First Output is generated.

The Scheduler's Objective: Maximize overall CPU throughput and utilization while strictly minimizing queue waiting time and ensuring fairness.

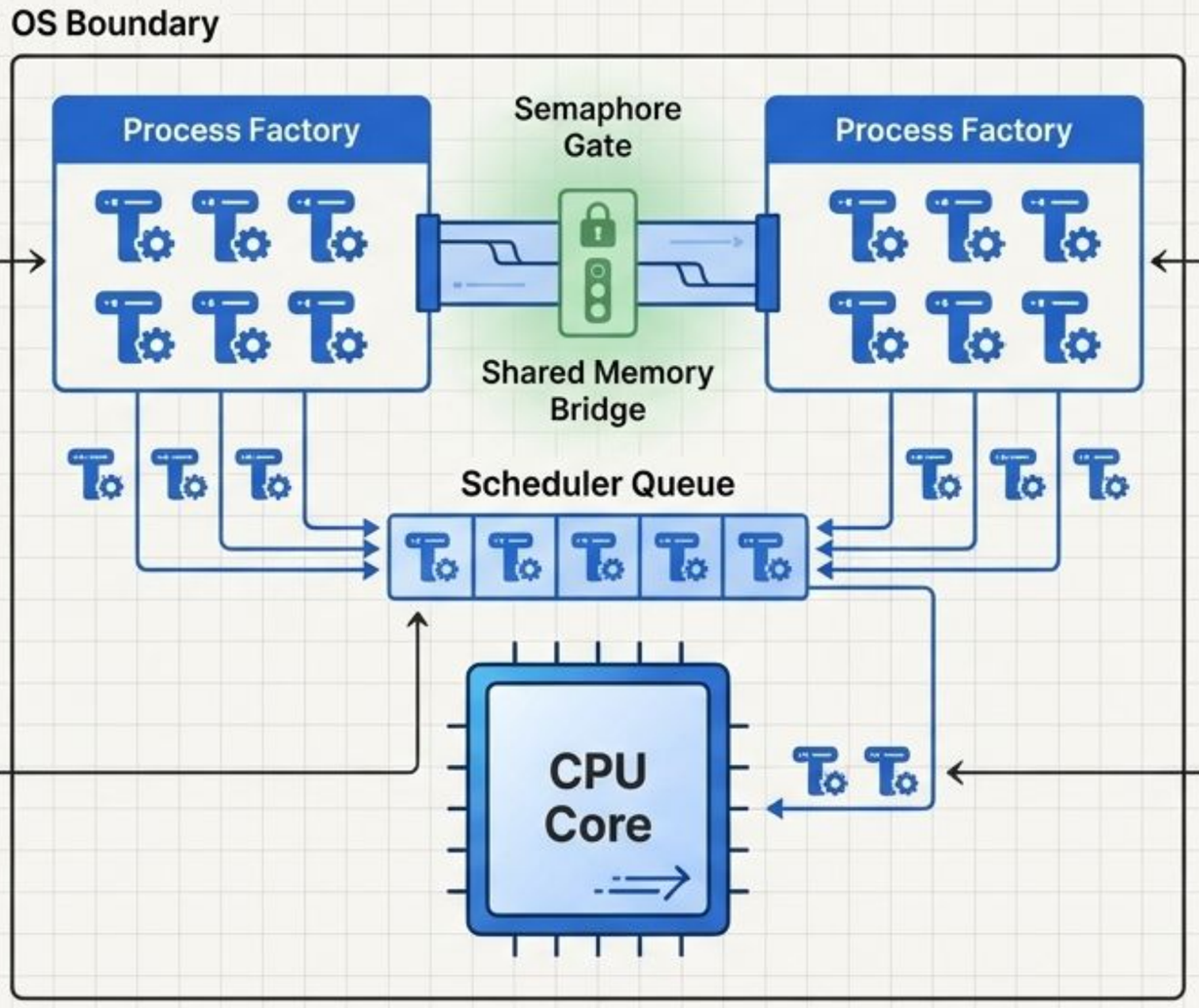
The OS Ecosystem Unified

1. Execution

Static programs spin up into dynamic Processes, subdivided into highly efficient lightweight **Threads**.

2. Cooperation

Isolated processes bridge the gap using IPC (Shared Memory or Message Passing) to share critical data.



3. Safety

Critical Sections are strictly guarded by atomic **Semaphores** to eradicate data collisions and Race Conditions.

4. Management

The OS **Scheduler** masterminds the queue, preemptively balancing CPU time to maximize system throughput.